

Microsoft Fabric에서 API for GraphQL 작업하기

Microsoft Fabric의 API for GraphQL은 널리 채택되고 익숙한 API 기술을 사용하여 여러 데이터 소스를 빠르고 효율적으로 쿼리할 수 있게 해주는 데이터 액세스 계층입니다. 이 API를 사용하면 백엔드 데이터 소스의 세부 사항을 추상화하여 애플리케이션의 로직에 집중할 수 있으며, 클라이언트가 필요로 하는 모든 데이터를 단일 호출로 제공할 수 있습니다. GraphQL은 간단한 쿼리 언어와 쉽게 조작할 수 있는 결과 집합을 사용하여 애플리케이션이 Fabric의 데이터에 액세스하는 데 걸리는 시간을 최소화합니다.

이 실습을 완료하는 데는 약 **30분**이 소요됩니다.

Note: 이 실습을 완료하려면 [Microsoft Fabric 평가판](#)이 필요합니다.

개념 설명: API for GraphQL이란 무엇일까요?

GraphQL은 API(Application Programming Interface)를 위한 쿼리 언어이자, 기존 데이터를 사용하여 이러한 쿼리를 수행하기 위한 서버 측 런타임입니다. 전통적인 REST API와는 다른 접근 방식을 제공합니다.

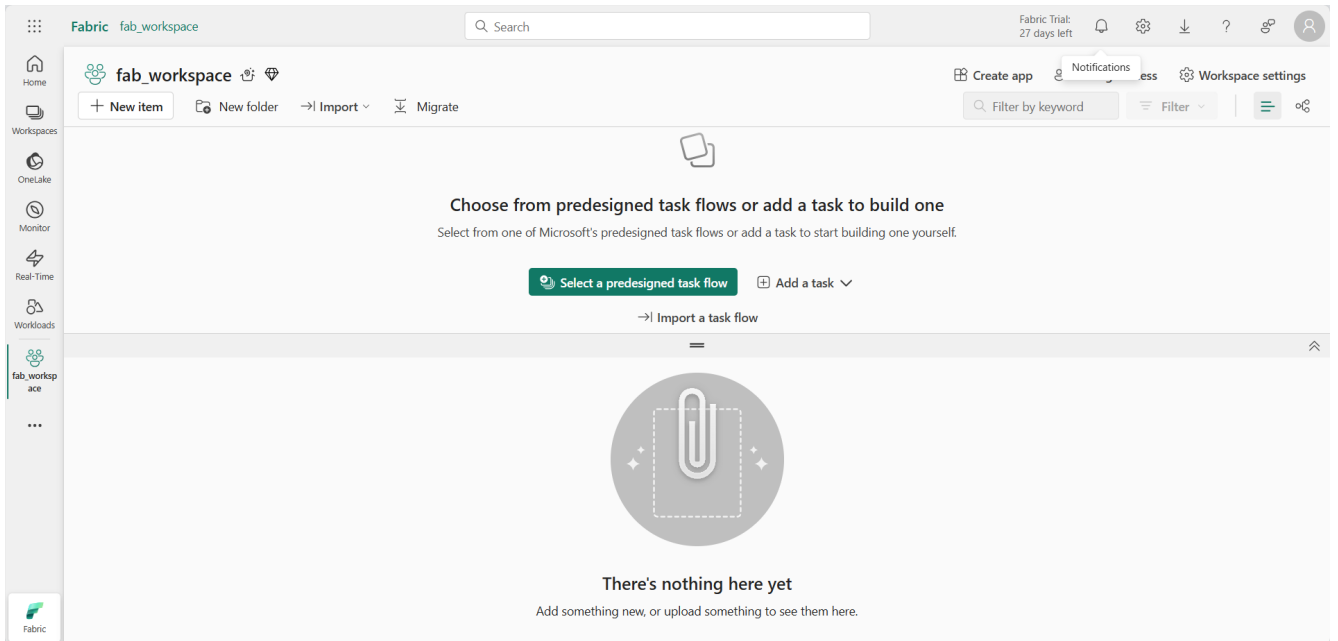
- **REST API의 한계:** 전통적인 REST API에서는 클라이언트(예: 웹 애플리케이션, 모바일 앱)가 데이터를 가져오기 위해 정해진 여러 개의 엔드포인트(URL)에 요청을 보내야 할 수 있습니다. 예를 들어, 제품 정보와 해당 제품의 카테고리 정보를 함께 가져오려면 `/products/{id}` 와 `/categories/{id}` 두 개의 엔드포인트를 호출해야 할 수 있습니다(이를 **언더페칭(under-fetching)** 이라고 합니다). 반대로, 단일 엔드포인트가 너무 많은 정보를 반환하여 클라이언트가 필요하지 않은 데이터까지 받게 될 수도 있습니다(이를 **오버페칭(over-fetching)** 이라고 합니다).
- **GraphQL의 해결책:** GraphQL은 단일 엔드포인트를 통해 클라이언트가 필요한 데이터의 구조를 정확하게 요청할 수 있게 해줍니다. 클라이언트는 쿼리를 통해 원하는 필드만 명시적으로 요청하고, 서버는 요청된 구조와 정확히 일치하는 JSON 응답을 반환합니다. 이로써 언더페칭과 오버페칭 문제를 해결하고 네트워크 사용량을 최적화할 수 있습니다.

Microsoft Fabric의 API for GraphQL은 Fabric 내의 데이터 소스(예: SQL Database, Lakehouse) 위에 이러한 GraphQL 계층을 손쉽게 생성할 수 있게 해주는 기능입니다. 복잡한 서버 코드 없이도 데이터 소스를 선택하기만 하면, Fabric이 자동으로 GraphQL 스키마와 엔드포인트를 생성해주어 개발자가 데이터를 매우 효율적으로 소비할 수 있도록 지원합니다.

Workspace 생성

Fabric에서 데이터를 다루기 전에, Fabric 평가판이 활성화된 Workspace(작업 영역)를 생성해야 합니다.

1. 브라우저에서 [Microsoft Fabric 홈페이지](https://app.fabric.microsoft.com/home?experience=fabric) (`https://app.fabric.microsoft.com/home?experience=fabric`) 로 이동하여 Fabric 자격 증명으로 로그인합니다.
2. 왼쪽 메뉴 바에서 **New workspace**를 선택합니다.
3. 원하는 이름으로 새 Workspace를 만들고, Fabric 용량을 포함하는 라이선스 모드(*Trial*, *Premium*, 또는 *Fabric*)를 선택합니다.
4. 새 Workspace가 열리면 비어 있어야 합니다.



샘플 데이터로 SQL Database 생성하기

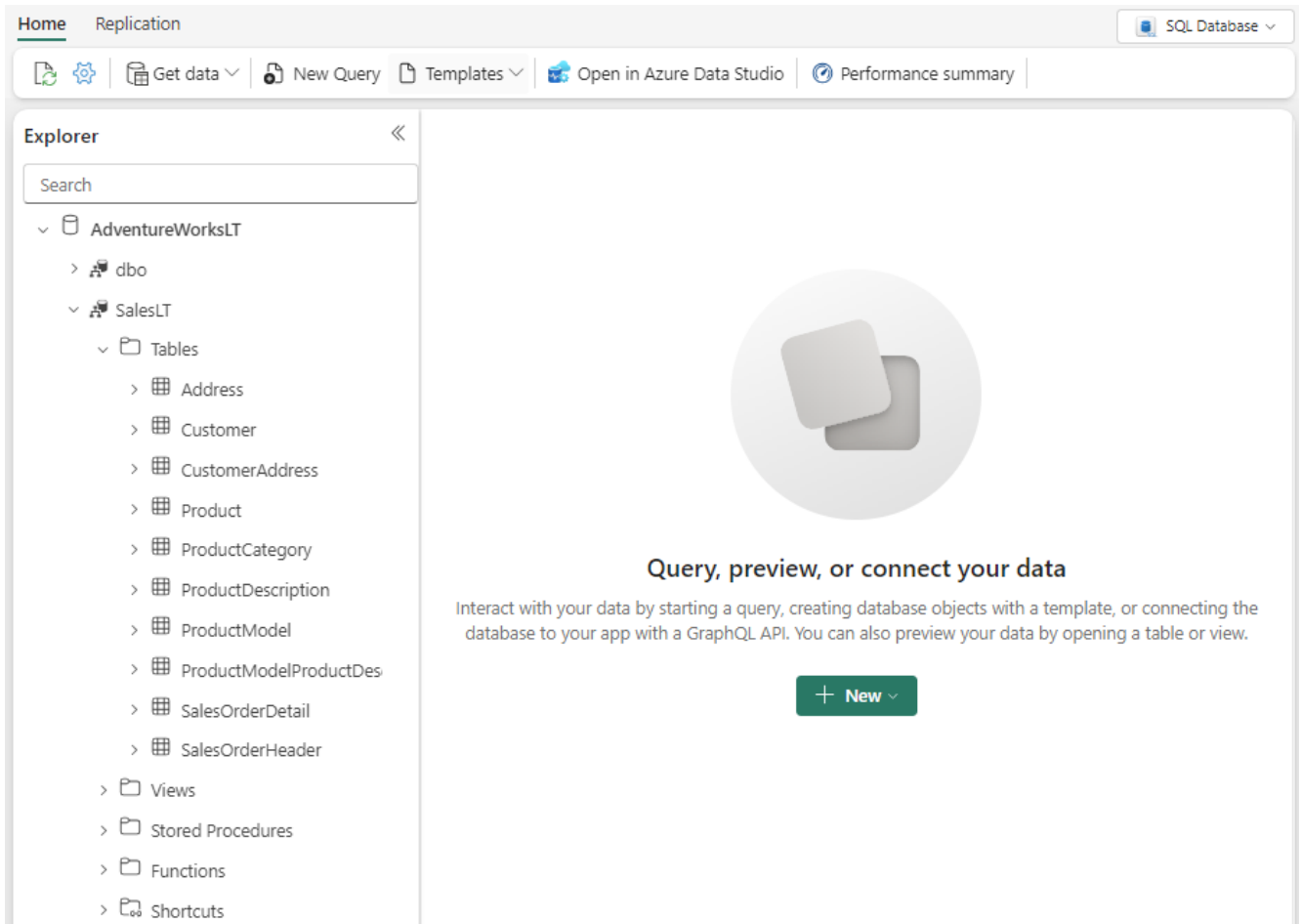
이제 Workspace가 있으므로 SQL Database를 생성할 차례입니다.

1. 왼쪽 메뉴 바에서 **Create**를 선택합니다. *New* 페이지의 *Databases* 섹션에서 **SQL database**를 선택합니다.

Note: **Create** 옵션이 사이드바에 고정되어 있지 않으면 먼저 줄임표(...) 옵션을 선택해야 합니다.

2. 데이터베이스 이름으로 **AdventureWorksLT**를 입력하고 **Create**를 선택합니다.
3. 데이터베이스가 생성되면 **Sample data** 카드에서 데이터베이스에 샘플 데이터를 로드할 수 있습니다.

약 1분 후, 시나리오에 맞는 샘플 데이터로 데이터베이스가 채워집니다.



SQL Database 쿼리하기

SQL 쿼리 편집기는 IntelliSense, 코드 완성, 구문 강조, 클라이언트 측 파싱 및 유효성 검사를 지원합니다. 데이터 정의 언어 (DDL), 데이터 조작 언어(DML), 데이터 제어 언어(DCL) 문을 실행할 수 있습니다.

1. **AdventureWorksLT** 데이터베이스 페이지에서 **Home**으로 이동하여 **New query**를 선택합니다.
2. 새로운 빈 쿼리 창에 다음 T-SQL 코드를 입력하고 실행합니다.

```
SELECT
    p.Name AS ProductName,
    pc.Name AS CategoryName,
    p.ListPrice
FROM
    SalesLT.Product p
INNER JOIN
    SalesLT.ProductCategory pc ON p.ProductCategoryID = pc.ProductCategoryID
ORDER BY
    p.ListPrice DESC;
```

코드 설명:

이 쿼리는 AdventureWorksLT 데이터베이스의 제품 데이터를 조회하는 표준 SQL 문입니다.

- `SELECT p.Name AS ProductName, pc.Name AS CategoryName, p.ListPrice`: `Product` 테이블에서 제품 이름(`Name`)을 `ProductName`으로, `ProductCategory` 테이블에서 카테고리 이름(`Name`)을 `CategoryName`으로, 그리고 `Product` 테이블에서 정가(`ListPrice`)를 선택하여 결과에 포함시킵니다. `AS`를 사용하여 열의 별칭(alias)을 지정했습니다.
- `FROM SalesLT.Product p INNER JOIN SalesLT.ProductCategory pc ON p.ProductCategoryID = pc.ProductCategoryID`: `Product` 테이블(별칭 `p`)과 `ProductCategory` 테이블(별칭 `pc`)을 조인합니다. 조인 조건은 두 테이블에 공통으로 존재하는 `ProductCategoryID` 열을 기준으로 하며, 이를 통해 각 제품이 어떤 카테고리에 속하는지 연결합니다.
- `ORDER BY p.ListPrice DESC`: 최종 결과를 제품의 정가(`ListPrice`) 기준으로 내림차순(`DESC`)으로 정렬합니다. 즉, 가장 비싼 제품이 맨 위에 표시됩니다.

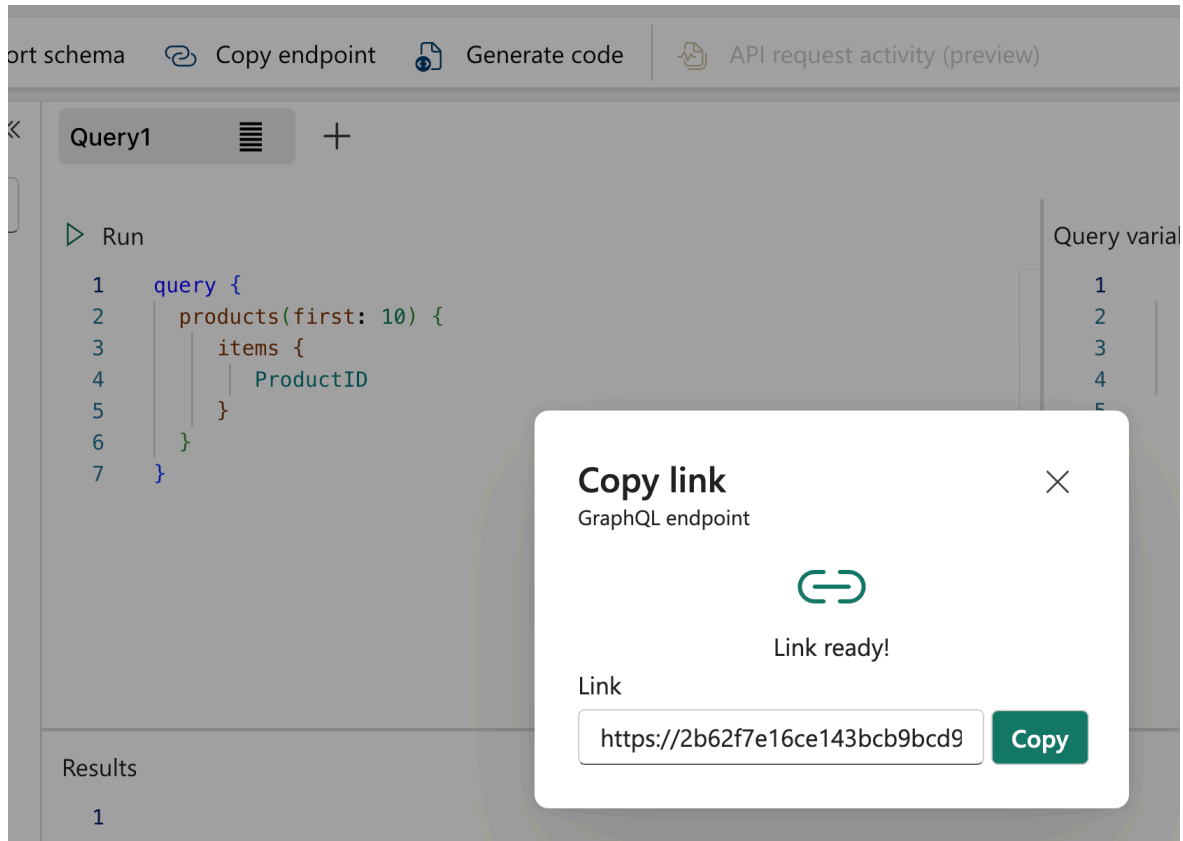
이 쿼리는 `Product`와 `ProductCategory` 테이블을 조인하여 제품 이름, 카테고리, 그리고 정가를 가격 내림차순으로 정렬하여 표시합니다.

3. 모든 쿼리 탭을 닫습니다.

API for GraphQL 생성하기

먼저, 판매 주문 데이터를 노출하기 위해 GraphQL 엔드포인트를 설정합니다. 이 엔드포인트를 사용하면 날짜, 고객, 제품과 같은 다양한 매개변수를 기반으로 판매 주문을 쿼리할 수 있습니다.

1. Fabric 포털에서 Workspace로 이동하여 **+ New item**을 선택합니다.
2. **Develop data** 섹션으로 이동하여 **API for GraphQL**을 선택합니다.
3. 이름을 입력하고 **Create**를 선택합니다.
4. API for GraphQL의 메인 페이지에서 **Select data source**를 선택합니다.
5. 연결 옵션을 선택하라는 메시지가 표시되면 **Connect to Fabric data sources with single sign-on(SSO) authentication**을 선택합니다.
6. **Choose the data you want to connect** 페이지에서 이전에 생성한 `AdventureWorksLT` 데이터베이스를 선택합니다.
7. **Connect**를 선택합니다.
8. **Choose data** 페이지에서 `SalesLT.Product` 테이블을 선택합니다.
9. 데이터를 미리 보고 **Load**를 선택합니다.
10. **Copy endpoint**를 선택하고 공개 URL 링크를 확인합니다. 지금은 필요 없지만, API 주소를 복사해야 할 때 이곳을 사용하면 됩니다.



Mutations 비활성화하기

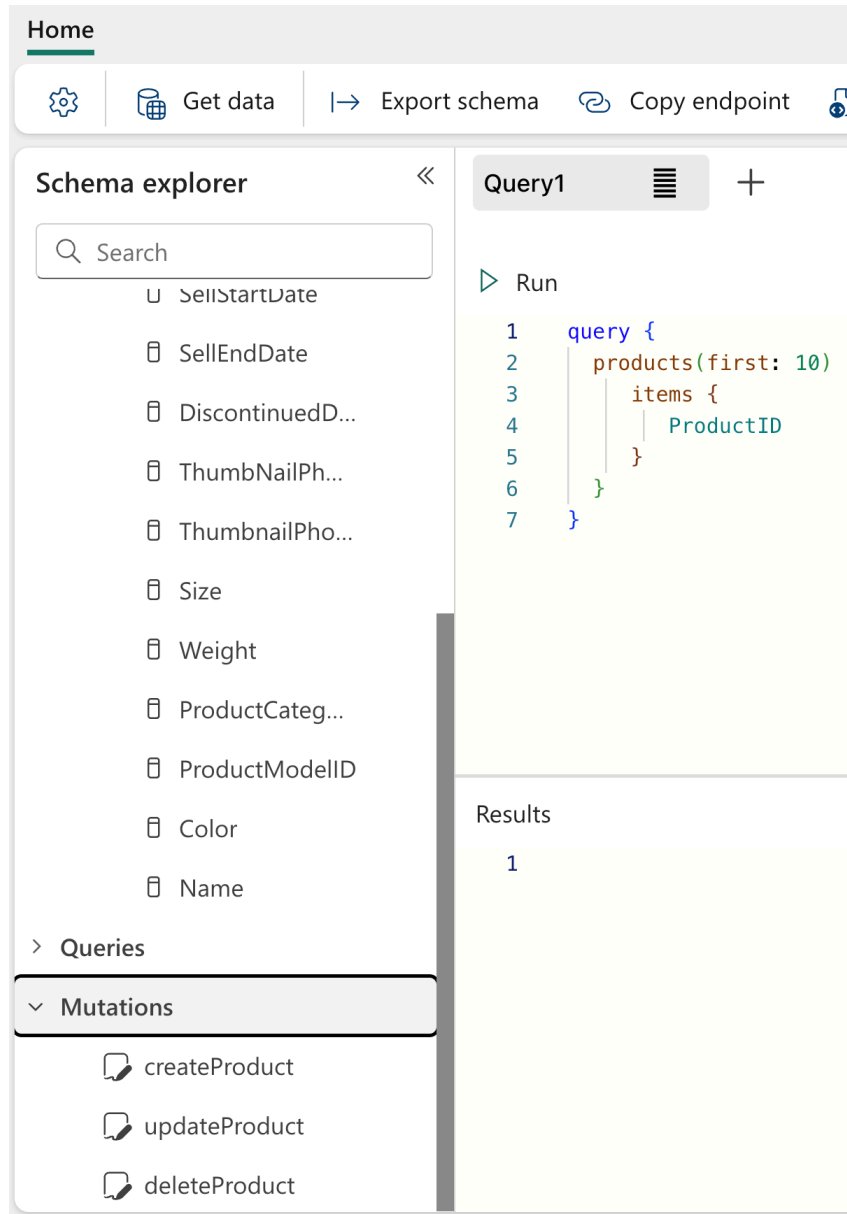
개념 설명: Queries vs. Mutations

GraphQL API는 크게 두 가지 유형의 작업을 정의합니다.

- **Queries (쿼리):** 데이터를 읽는 작업입니다. SQL의 `SELECT` 문과 유사합니다. 클라이언트는 쿼리를 사용하여 서버로부터 데이터를 조회합니다.
- **Mutations (뮤테이션):** 데이터를 쓰는(생성, 수정, 삭제하는) 작업입니다. SQL의 `INSERT`, `UPDATE`, `DELETE` 문과 같습니다. 클라이언트는 뮤테이션을 사용하여 서버의 데이터를 변경합니다.
- Fabric에서 `API for GraphQL`을 생성하면 기본적으로 선택된 테이블에 대해 데이터를 조회하는 `Query`와 데이터를 생성, 업데이트, 삭제하는 `Mutation`이 자동으로 만들어집니다. 하지만 많은 시나리오에서 API를 통해 데이터를 변경하는 것을 원치 않고, 오직 조회용으로만 제공하고 싶을 수 있습니다. 이럴 때 **Mutations**를 비활성화하여 API를 안전한 **읽기 전용 (read-only)**으로 만들 수 있습니다.

API가 생성되었으므로, 이 시나리오에서는 판매 데이터를 읽기 작업에만 노출하려고 합니다.

1. API for GraphQL의 **Schema explorer**에서 **Mutations**를 확장합니다.



2. 각 뮤테이션 옆의 ... (줄임표)를 선택하고 **Disable**을 선택합니다.

이렇게 하면 API를 통한 데이터 수정이나 업데이트가 방지됩니다. 즉, 데이터는 읽기 전용이 되며, 사용자는 데이터를 보거나 쿼리할 수만 있고 변경할 수는 없습니다.

GraphQL을 사용하여 데이터 쿼리하기

이제 GraphQL을 사용하여 이름이 "HL Road Frame"으로 시작하는 모든 제품을 찾아보겠습니다.

1. GraphQL 쿼리 편집기에 다음 쿼리를 입력하고 실행합니다.

```

query {
  products(filter: { Name: { startsWith: "HL Road Frame" } }) {
    items {
      ProductModelID
      Name
      ListPrice
      Color
      Size
      ModifiedDate
    }
  }
}

```

Schema explorer

- ☐ ListPrice
- ☐ rowguid
- ☐ ModifiedDate
- ☐ SellStartDate
- ☐ SellEndDate
- ☐ DiscontinuedD...
- ☐ ThumbNailPh...
- ☐ ThumbnailPho...
- ☐ Size
- ☐ Weight
- ☐ ProductCateg...
- ☐ ProductModelID
- ☐ Color
- ☐ Name

> Queries

Query1

Query2



Run

```

1 query {
2   products(filter: { Name: { startsWith: "HL Road Frame" } }) {
3     items {
4       ProductModelID
5       Name
6       ListPrice
7       Color
8       Size
9       ModifiedDate
10    }
11  }

```

Query variables

```

1
2 {
3
4 }
5

```

Results

```

32   "ListPrice": 1431.5000,
33   "Color": "Red",
34   "Size": "44",
35   "ModifiedDate": "2008-03-11T10:01:36.827Z"
36 },
37 {
38   "ProductModelID": 6,
39   "Name": "HL Road Frame - Red, 48",
40   "ListPrice": 1431.5000,
41   "Color": "Red",
42   "Size": "48",
43   "ModifiedDate": "2008-03-11T10:01:36.827Z"
44 },

```

코드 설명:

이 코드는 GraphQL 쿼리 언어의 구문을 따릅니다.

- `query { ... }`: 이것은 우리가 수행할 작업이 데이터를 읽는 `query`임을 명시합니다. (생략 가능하지만 명시적으로 적는 것이 좋습니다.)
- `products(...)`: 우리가 쿼리할 대상을 지정합니다. Fabric이 `SalesLT.Product` 테이블을 기반으로 자동으로 생성한 `products` 타입을 쿼리하겠다는 의미입니다.
- `(filter: { Name: { startsWith: "HL Road Frame" } })`: 쿼리할 대상에 대한 필터 조건을 지정하는 인수(argument)입니다.
 - `filter`: 필터링을 위한 객체입니다.
 - `Name: { ... }`: `Name` 필드에 대한 조건을 정의합니다.
 - `startsWith: "HL Road Frame"`: `Name` 필드의 값이 "HL Road Frame"으로 시작하는 항목만 필터링합니다. 이는 SQL의 `WHERE Name LIKE 'HL Road Frame%'`와 유사한 역할을 합니다.

- `{ items { ... } }`: 서버로부터 반환받을 데이터의 구조를 정의합니다. 이것이 GraphQL의 가장 큰 특징입니다.
 - `items`: 결과 목록을 나타냅니다.
 - `{ ProductModelID, Name, ... }`: 결과에 포함될 필드를 명시적으로 나열합니다. 클라이언트는 정확히 여기에 명시된 필드들만 받게 되므로, 불필요한 데이터를 전송받지 않습니다.

이 쿼리에서 `products`는 메인 타입이며, `ProductModelID`, `Name`, `ListPrice`, `Color`, `Size`, `ModifiedDate` 필드를 포함합니다. 이 쿼리는 이름이 *"HL Road Frame"*으로 시작하는 제품 목록을 반환할 것입니다.

추가 정보: 플랫폼에서 사용 가능한 다른 구성 요소에 대해 더 알아보려면 Microsoft Fabric 문서의 [What is Microsoft Fabric API for GraphQL?](#)를 참조하세요.

이 실습에서는 Microsoft Fabric에서 GraphQL을 사용하여 SQL 데이터베이스의 데이터를 생성, 쿼리 및 노출했습니다.

리소스 정리

데이터베이스 탐색을 마쳤다면, 이 실습을 위해 생성한 Workspace를 삭제할 수 있습니다.

1. 왼쪽 바에서 Workspace 아이콘을 선택하여 포함된 모든 항목을 봅니다.
2. 툴바의 ... 메뉴에서 **Workspace settings**를 선택합니다.
3. **General** 섹션에서 **Remove this workspace**를 선택합니다.